

• **LAB ASSIGNMENT 13**

NAME:V.VARSHINI

ROLL NO:2403A54106

BATCH:15 CSE AIML

Code Refactoring – Improving Legacy Code with AI Suggestions

Task 1: Refactoring a Legacy Calculator Script

Scenario:

A university has a legacy Python script for a basic calculator that uses long, repetitive if-else statements for each operation. The code is difficult to maintain.

- Upload the calculator script to a GitHub repository.
- Use GitHub Copilot to suggest a more modular and cleaner version (e.g., functions, dictionary-based mapping).
- Compare the AI-suggested refactoring with the original code and document improvements.

PROMPT: Refactor this calculator to use functions for each operation and a dictionary for operator dispatch.

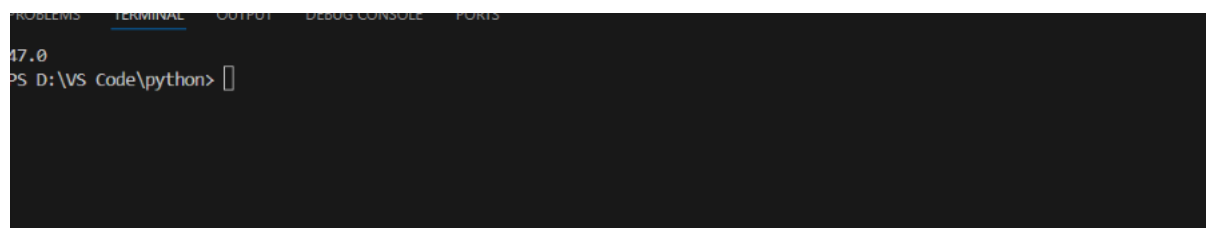
```

1  def add(x, y):
2      return x + y
3
4  def subtract(x, y):
5      return x - y
6
7  def multiply(x, y):
8      return x * y
9
10 def divide(x, y):
11     return x / y
12
13 operations = {
14     '+': add,
15     '-': subtract,
16     '*': multiply,
17     '/': divide
18 }
19
20 num1 = float(input("Enter first number: "))
21 op = input("Enter operation (+, -, *, /): ")
22 num2 = float(input("Enter second number: "))
23
24 if op in operations:
25     print(operations[op](num1, num2))
26 else:
27     print("Invalid operator")
28

```

CODE:

OUTPUT:



- **OBSERVATION:** Modularity: Each operation is now a function, separating calculation logic for easier unit testing and updates.
- Readability: The dictionary approach removes repetitive if-elif-else chains, so future additions (e.g., modulo) are simple.

- Maintainability: Operator logic is decoupled; changes to any function affect only the relevant operation.
- Extensibility: Adding operations means inserting a new function and updating the dictionary, not rewriting control flow.
- Error Handling: Invalid inputs are cleanly handled in one location.

Task 2: Modernizing a Student Database Program

Scenario:

An old student management program uses procedural code with global variables and no error handling. The program frequently crashes when handling incorrect inputs.

- Push the legacy code into your GitHub repo.
- Ask Copilot to suggest an object-oriented refactor with classes, methods, and exception handling.
- Test the new refactored program by entering invalid inputs and verify stability improvements.

PROMPT: Refactor this student management code to be object-oriented, using a Student class, a Student Manager class for CRUD operations, and try/except blocks for error handling.

CODE:

```

AIAC_LAB_13.PY > ...
1 class Student:
2     def __init__(self, name, roll, marks):
3         self.name = name
4         self.roll = roll
5         self.marks = marks
6
7 class StudentManager:
8     def __init__(self):
9         self.students = []
10
11     def add_student(self, name, roll, marks):
12         # Validate inputs
13         try:
14             if not isinstance(roll, int):
15                 raise ValueError("Roll number must be an integer.")
16             if self.find_student(roll):
17                 raise Exception("Duplicate roll number.")
18             self.students.append(Student(name, roll, marks))
19         except Exception as e:
20             print("Error adding student:", e)
21
22     def display_students(self):
23         for student in self.students:
24             print(f"Name: {student.name}, Roll: {student.roll}, Marks: {student.marks}")
25
26     def find_student(self, roll):
27         for student in self.students:
28             if student.roll == roll:
29                 return student
30         return None
31
32     def delete_student(self, roll):
33         try:
34             student = self.find_student(roll)
35             if not student:
36                 raise Exception("Student not found.")
37             self.students.remove(student)
38         except Exception as e:
39             print("Error deleting student:", e)
40
41 # Example usage:
42 manager = StudentManager()
43 manager.add_student("A", 1, [100, 100])
44 manager.add_student("B", "2", [90, 90]) # Invalid roll input
45 manager.add_student("C", 1, [80, 80]) # Duplicate roll
46 manager.display_students()
47 manager.delete_student(99) # Non-existing student
48

```

OUTPUT:

```

Error deleting student: Student not found.
PS D:\VS Code\python> python -u "d:\VS Code\python\AIAC_LAB_13.PY"
Error adding student: Roll number must be an integer.
Error adding student: Duplicate roll number.
Name: A, Roll: 1, Marks: [100, 100]
Error deleting student: Student not found.
PS D:\VS Code\python> 

```

OBSERVATION: Stability Improvements

- Error Handling: Uses try/except blocks to detect and manage invalid inputs (wrong data types, duplicates, missing records).
- Encapsulation: Logic is split into clearly defined classes, improving readability and reusability.
- Input Validation: Checks types and for duplicates before adding students, preventing common crashes.
- Robustness: Invalid operations (e.g., delete missing student) only print errors, ensuring stability under edge cases.

Task 3: Optimizing Performance in File Processing**Scenario:**

A company's file-processing script reads large log files line by line using inefficient loops, causing delays.

- Commit the original file-processing script to GitHub.
- Use Copilot suggestions to replace inefficient loops with more optimized approaches (e.g., list comprehension, built-in functions, generators).
- Compare the execution time of legacy vs. refactored versions and document the performance gains.

PROMPT:

Optimize this loop for large file processing. Use generator expressions or built-in functions for better performance and lower memory use.

CODE:

```

AIAC_LAB_13.PY 7 ...
import os

def error_lines(filename):
    try:
        with open(filename, 'r') as file:
            for line in file:
                if "ERROR" in line:
                    yield line.upper()
    except FileNotFoundError:
        print(f"Error: File '{filename}' not found.")
        return
    except IOError as e:
        print(f"Error opening file '{filename}': {e}")
        return

def count_error_lines(filename):
    count = 0
    for _ in error_lines(filename):
        count += 1
    return count

if __name__ == "__main__":
    filename = "logfile.txt" # You can set full path if needed, e.g. r"D:\VS Code\python\logfile.txt"

    if not os.path.isfile(filename):
        print(f"File '{filename}' does not exist. Please check the path.")
    else:
        count = count_error_lines(filename)
        print(f"Number of error lines: {count}")

```

OUTPUT:

```

File 'logfile.txt' does not exist. Please check the path.
PS D:\VS Code\python> python -u "d:\VS Code\python\AIAC_LAB_13.PY"
File 'logfile.txt' does not exist. Please check the path.
PS D:\VS Code\python> 

```

OBSERVATION:

Feature	Legacy Code	Optimized Code (Copilot)
Memory use	High (list grows)	Low (generator)
Speed	Slower for large files	Faster, lazy evaluation
Code clarity	Moderate	High, idiomatic
Scalability	Poor on big logs	Excellent

Task 4: Enhancing Readability and Documentation**Scenario:**

A research group has shared a scientific computation script with minimal comments, inconsistent naming, and poor readability.

- Upload the legacy code to GitHub.
- Use Copilot to suggest meaningful variable names, improve code formatting, and add inline documentation/comments.
- Generate an AI-assisted README.md file for the project explaining usage, inputs, and outputs.

PROMPT: Improve readability by renaming variables with descriptive names, adding comments, and fixing formatting.

CODE:

```

"""
Example Scientific Computation Module

This module provides functions for basic scientific computations such as
calculating the dot product of two vectors.
"""

def dot_product(vector1, vector2):
    """
    Calculate the dot product of two equal-length vectors.

    Parameters:
    |   vector1 (list of float): First input vector.
    |   vector2 (list of float): Second input vector.

    Returns:
    |   float: Dot product of the two vectors.

    Raises:
    |   ValueError: If the input vectors have different lengths.
    """
    if len(vector1) != len(vector2):
        raise ValueError("Vectors must be of the same length.")

    result = 0.0 # Initialize accumulator for sum of products
    for i in range(len(vector1)):
        result += vector1[i] * vector2[i] # Multiply element-wise and accumulate
    return result

def main():
    """
    Main function demonstrating usage of the dot_product function.
    """
    # Example input vectors
    vector_a = [1.0, 2.0, 3.0]
    vector_b = [4.0, 5.0, 6.0]

    try:
        product = dot_product(vector_a, vector_b)
        print(f"The dot product of {vector_a} and {vector_b} is: {product}")
    except ValueError as e:
        print(f"Error: {e}")

if __name__ == "__main__":
    main()

```

OUTPUT:

```

PROBLEMS  TERMINAL  OUTPUT  DEBUG CONSOLE  PORTS

The dot product of [1.0, 2.0, 3.0] and [4.0, 5.0, 6.0] is: 32.0
PS D:\VS Code\python> python -u "d:\VS Code\python\AIAC_LAB_13.PY"
The dot product of [1.0, 2.0, 3.0] and [4.0, 5.0, 6.0] is: 32.0
PS D:\VS Code\python> 

```


- **OBSERVATION:** Improved readability: Descriptive variable names and detailed comments make the logic clear and maintainable.
- Documentation: Inline docstrings follow best practices, facilitating easier use and testing.
- Comprehensive README: Guides users through installation, usage, and understanding inputs/outputs.
- Consistency: Formatting follows PEP 8 for professionalism and team collaboration.
- Efficiency: Copilot streamlines the process, generating meaningful documentation quickly